In Search of an Understandable Consensus Algorithm (Raft

Diego Ongaro & John Ousterhout - Stanford University (USENIX ATC 2014)

Overview & Design Philosophy

Raft is a consensus algorithm for managing a replicated log. It produces a result equivalent to (multi-)Paxos, and it is as efficient as Paxos, but its structure is different, which makes Raft more understandable and easier to implement in practical distributed engines.

Core Decomposed Subproblems

Leader Election: A new leader must be chosen when an existing leader fails.
Log Replication: The leader must accept log entries from clients and replicate them across the cluster.
Safety: If any server has applied a log entry at a given index, no other server will apply a different entry.

Raft Invariants & State Transition

1. Election Safety: At most one leader can be elected in a given term.
2. Leader Append-Only: A leader never overwrites or truncates its log; it only appends new entries.
3. Log Matching: If two logs contain an entry with the same index and term, then the logs are identical up to that index.
4. Leader Completeness: If a log entry is committed in a given term, it is present in the logs of all leaders for higher terms.

RPC Interfaces

RequestVote: Invoked by candidates during elections to gather quorum votes.
AppendEntries: Invoked by leaders to replicate log entries and as periodic heartbeats.